

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF REPUBLIC OF MOLDOVA
TECHNICAL UNIVERSITY OF MOLDOVA
FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE ENGINEERING AND AUTOMATICS

IncidentFlow: A Domain-Specific Language for Cybersecurity Incident Reporting

Project report

Mentor: prof., Brânzan Leon

Students: Kushnirenko Ekaterina, FAF-243

Felicia Ojog, FAF-243

Botezatu Gabriela, FAF-243

Șoimu Ionuț, FAF-243

Chișinău, 2026

Abstract

This paper presents IncidentFlow, a domain-specific language (DSL) for cybersecurity incident response documentation and automation. In prevailing practice, response procedures are maintained as static documents and executed manually under severe time constraints, forcing teams to repeatedly check mandatory sections, formatting constraints, and compliance evidence while incidents are still unfolding. As established in Chapter 1, this operational pattern contributes to procedural omissions, coordination latency, and weakened auditability.

The core problem is that incident response playbooks remain largely static, difficult to validate before real incidents, and disconnected from executable workflows. Existing automation ecosystems provide partial remediation but are frequently constrained by vendor lock-in, elevated adoption cost, and limited support for transparent version-controlled review workflows. Consequently, organizations lack a practical mechanism that is simultaneously human-readable for responders and machine-executable for dependable automation.

The proposed solution encodes incident logic and reporting semantics in a unified formal specification, including triggers, phases, control-flow constructs, verification handlers, severity transitions, and reporting actions. This design reduces repetitive requirement-checking by making structural completeness and procedural consistency properties of the language model itself.

Implementation is realized through an ANTLR-based parsing pipeline comprising a formal grammar, domain-constrained lexical analysis, and deterministic parser behavior with explicit error instrumentation, as detailed in Chapter 2. This architecture furnishes a syntactically validated intermediate representation that can be leveraged for downstream automation, conformance assurance, and reproducible report synthesis.

The principal contribution is an extensible foundation for process-aware incident reporting that improves operational efficiency, attenuates human error, and strengthens evidentiary quality in high-pressure security operations.

Keywords: Domain-Specific Languages, Cybersecurity Incident Response, Incident Reporting Automation, ANTLR, Parser Design, Compliance Engineering, Human Error Reduction.

Content

Introduction	4
Domain Analysis and Problem Framing	5
1.1 Problem Description and Problem Analysis	5
1.1.1 Problem Statement	5
1.1.2 Domain Overview: Cybersecurity Incident Response	5
1.2 Description of the DSL	10
Language Architecture	11
2.1 Grammar Structure	11
2.2 Lexer Rules	14
2.3 Parser Logic	16
2.4 Derivation Tree Analysis	18
Implementation	20
3.0.1 Command-Line Interface (CLI)	20
3.0.2 HTTP API Backend	22
3.0.3 Frontend Architecture and UI	23
Conclusions	27
Appendix	28
3.1 Additional UI Views	47
Bibliography	48

Introduction

Cybersecurity incidents are handled under pressure, but incident reports are still often produced with manual, template-driven work. Analysts and incident leads must investigate technical evidence, coordinate decisions, and document actions in parallel, while also making sure the final report follows predefined structures and mandatory fields. In practice, this creates a recurring burden: teams repeatedly pause to check whether they have respected all template requirements, formatting rules, and reporting constraints. The process becomes slow, error-prone, and hard to sustain during active incidents.

This project focuses on improving that situation through usability and automation. The core idea is that reporting should be generated from the incident-handling workflow itself, not written from scratch after the response ends. When key actions, timestamps, decisions, and ownership are captured in structured form during execution, the system can automatically assemble most of the report content. This reduces duplicate work, keeps information consistent across sections, and allows teams to spend less time verifying checklist items in static templates.

An important objective is to reduce the constant need to manually validate compliance with report templates. Instead of repeatedly checking if each required section exists and whether every mandatory detail has been included, responders should receive built-in guidance that enforces structure as they work. In this approach, template requirements are embedded into the flow, so completeness and consistency become default outcomes rather than late-stage corrections. The benefit is both operational and organizational: faster reporting cycles, fewer omissions, and clearer artifacts for management and audit use.

The project also emphasizes ease of use for different roles involved in incident response. A usable system should make the next step explicit, minimize unnecessary data entry, and preserve human judgment where it matters most. Automation is treated as support, not replacement: it handles repetitive formatting and report composition tasks, while analysts remain responsible for interpretation, prioritization, and decision-making. This balance is essential for producing reports that are both technically reliable and understandable to non-technical stakeholders.

The rest of the report follows this direction. The next chapter analyzes the incident response domain and identifies the procedural and reporting gaps that motivate the proposed solution. It then introduces the initial concept for executable guidance and automated report generation. The final chapter summarizes the current results and outlines the next steps. Overall, the project aims to make incident reporting accurate, timely, and significantly easier by integrating template compliance and report generation directly into daily response workflows.

Domain Analysis and Problem Framing

1.1 Problem Description and Problem Analysis

1.1.1 Problem Statement

Incident response playbooks are structured, procedural documents that prescribe the exact steps an analyst must follow when handling a specific category of incident, such as ransomware, phishing, account compromise, or data exfiltration. Playbooks operationalize the high-level guidance of frameworks like NIST SP 800-61 into actionable, role-specific instructions. They typically include decision trees, escalation criteria, specific system commands, communication templates, and approval gates [1].

Incident response playbooks are currently static, weakly testable documents that cannot be validated or executed automatically with sufficient rigor, resulting in slower, less consistent, and more error-prone response during cybersecurity incidents.

1.1.2 Domain Overview: Cybersecurity Incident Response

Cybersecurity incident response (IR) is one of the most operationally demanding disciplines within the broader field of information security. As organizations increasingly depend on digital infrastructure, the frequency, sophistication, and financial impact of cyberattacks have grown substantially. According to IBM's *Cost of a Data Breach Report 2025*, the global average cost of a data breach remains a major organizational concern [2]. This environment places enormous pressure on security teams to detect, analyze, and recover from incidents in a structured, repeatable, and auditable manner. The domain of incident response sits at the intersection of technical operations, organizational governance, and regulatory compliance, making it inherently multidisciplinary and complex.

What Is Incident Response?

Incident response is the organized approach to addressing and managing the aftermath of a security breach or cyberattack. The primary goals are to limit damage, reduce recovery time and costs, preserve forensic evidence, and prevent future recurrences [3]. A *security incident* is formally defined by NIST as “a violation or imminent threat of violation of computer security policies, acceptable use policies, or standard security practices” [3]. This definition encompasses a wide range of events, from malware infections and ransomware deployments to insider threats, data exfiltration, and distributed denial-of-service (DDoS) attacks.

The discipline is governed by several internationally recognized frameworks and standards. The most widely adopted in North America is NIST Special Publication 800-61 *Computer Security Incident Handling Guide*; Rev. 2 was withdrawn on April 03, 2025 and superseded by Rev. 3 [3, 4]. In Europe and internationally, ISO/IEC 27035 *Information Security Incident Management* provides a comparable struc-

ture methodology [5]. The SANS Institute additionally publishes its own IR methodology and maintains the widely referenced *Incident Handler's Handbook*, which is extensively used for practitioner training [1]. While these frameworks differ in scope and terminology, they share a common structural philosophy: incident response must be lifecycle-based, documented, and continuously improved.

The Incident Response Lifecycle (NIST SP 800-61)

NIST SP 800-61 defines the incident response lifecycle as consisting of four primary phases, each with distinct objectives and activities [3]. These phases are not always strictly sequential; in practice, responders may cycle between phases or execute certain activities in parallel depending on the nature and scope of the incident.

Phase 1: Preparation. The preparation phase encompasses all activities that take place before an incident occurs. This includes establishing incident response policies and procedures, defining team roles and responsibilities, procuring and configuring response tooling, creating and maintaining *playbooks* (step-by-step procedural guides for specific incident types), setting up communication and escalation plans, and conducting tabletop exercises and simulations. NIST emphasizes that preparation is arguably the most critical phase because the quality of preparation directly determines the effectiveness of all subsequent phases [3]. Organizations that lack mature preparation capabilities tend to exhibit significantly longer mean time to detect (MTTD) and mean time to respond (MTTR) metrics. According to IBM's 2025 report, organizations with mature IR practices consistently reduce breach impact compared to organizations without tested response capabilities [2].

Phase 2: Detection and Analysis. The detection and analysis phase involves identifying potential security events and determining whether they constitute actual incidents. Detection sources include Security Information and Event Management (SIEM) platforms, Intrusion Detection and Prevention Systems (IDS/IPS), endpoint detection and response (EDR) tools, network traffic analysis, user reports, and third-party threat intelligence feeds [3]. The analytical challenge at this phase is substantial: Security Operations Center (SOC) analysts are routinely exposed to hundreds or thousands of alerts per shift, the overwhelming majority of which are false positives. A 2022 study by Ponemon Institute found that SOC analysts spend substantial time investigating alerts that turn out to be non-incidents, contributing to analyst burnout and reduced organizational security posture [6]. Effective detection requires not only robust tooling but well-defined triage procedures and severity classification criteria, typically organized into tiered priority levels (P1 through P4 or similar) that guide how quickly and intensively analysts must respond.

Phase 3: Containment, Eradication, and Recovery. Once an incident is confirmed and analyzed, the response team must act to limit its spread (containment), remove the threat actor and all malicious artifacts from affected systems (eradication), and restore normal operations from a known-good state (recovery) [3]. Containment strategies range from short-term isolation of individual hosts to full network segmentation, and

the choice depends heavily on the incident type and the organization's operational tolerance for disruption. Eradication may involve reimaging systems, revoking compromised credentials, patching exploited vulnerabilities, and removing persistence mechanisms. Recovery involves restoring services from clean backups, verifying system integrity, and gradually returning systems to production under close monitoring. A critical challenge throughout this phase is the tension between speed and thoroughness: moving too quickly risks missing residual attacker footholds, while excessive caution prolongs business disruption [1].

Phase 4: Post-Incident Activity. The final phase encompasses all activities that occur after the incident has been resolved. These include conducting a formal lessons-learned review, updating IR playbooks and detection rules based on findings, producing regulatory and management reports, preserving forensic evidence in accordance with legal requirements, and filing breach notifications where mandated by law [3]. This phase is frequently underinvested in practice: a 2021 survey by ISACA found that fewer than 40% of organizations conducted formal post-incident reviews after every security incident [7]. The consequence is a failure to incorporate operational knowledge back into preparation materials, leading to repeated errors in future incidents.

The Role of Playbooks in Incident Response

In practice, playbooks are most commonly stored and distributed as static documents — PDF files, Word documents, or wiki pages — that analysts must read and manually follow under time pressure. This creates a significant operational risk: analyst workload and sustained alert pressure during high-stress incident response scenarios are associated with procedural drift, step skipping, and documentation gaps [6]. Furthermore, static playbooks are difficult to keep current; as threat landscapes evolve and organizational environments change, outdated procedures can mislead responders rather than guide them.

The concept of *executable playbooks* or *security orchestration, automation, and response* (SOAR) platforms has emerged as a technical response to these challenges. SOAR platforms such as Splunk SOAR (formerly Phantom), Palo Alto XSOAR, and IBM Resilient allow organizations to encode playbook logic into automated workflows that execute actions, track state, and enforce procedural compliance programmatically [8, 9, 10]. However, enterprise SOAR platforms are expensive, complex to deploy, and typically accessible only to large organizations with mature security operations, leaving small and medium-sized enterprises (SMEs) without viable alternatives to static document-based playbooks. In addition, many commercial ecosystems introduce platform dependency and limited portability, which complicates independent peer review, transparent version-controlled collaboration, and long-term procedural governance across heterogeneous toolchains.

Regulatory and Compliance Dimensions

Incident response does not operate in a legal vacuum. Organizations operating in or serving customers in regulated environments are subject to mandatory breach notification and incident documentation

requirements under a growing body of legislation and regulation.

The European Union’s *General Data Protection Regulation* (GDPR, Regulation 2016/679) requires organizations to notify the relevant supervisory authority within 72 hours of becoming aware of a personal data breach, and to notify affected individuals without undue delay where the breach is likely to result in high risk [11, 12, 13, 14]. The *Network and Information Security Directive* (NIS2 Directive, 2022/2555/EU), which came into force in October 2024, extends mandatory incident reporting obligations to a significantly broader range of sectors deemed critical infrastructure across EU member states, with substantial financial penalties for non-compliance [11, 12, 13, 14]. In the United States, sector-specific regulations such as the *Sarbanes-Oxley Act* (SOX) impose obligations on publicly traded companies to maintain accurate records of material cybersecurity events affecting financial reporting systems [11, 12, 13, 14]. The *Health Insurance Portability and Accountability Act* (HIPAA) similarly mandates breach notification and documentation requirements in the healthcare sector [11, 12, 13, 14].

A common thread across all of these regulatory frameworks is the requirement for *demonstrable, auditable evidence* that incidents were handled according to defined procedures. This creates a documentation burden that, under the pressure of an active incident, is frequently deprioritized in favor of immediate technical response actions. The resulting gap between what organizations claim their IR procedures are and what actually occurred during an incident is a recognized compliance risk, particularly during post-breach regulatory audits [7].

Stakeholders in Incident Response Operations

Effective incident response requires coordinated action across multiple organizational roles, each with distinct responsibilities, information needs, and operational pain points. Understanding the stakeholder landscape (as summarized in Table 1.1.1) is essential to identifying where operational gaps and inefficiencies arise within the IR domain.

The diversity of stakeholder needs highlights a structural challenge within the IR domain: different roles require different views and interfaces into the same underlying incident data, yet most organizations rely on a single shared artifact — the static playbook document — to coordinate all of these perspectives simultaneously.

Identified Gaps and Problem Areas in the Domain

Drawing on the preceding analysis of the IR lifecycle, playbook practices, regulatory requirements, and stakeholder needs, several recurring problem areas can be identified that represent systemic weaknesses within the cybersecurity incident response domain.

The first problem is the *auditability gap* that emerges at the intersection of operational IR and regulatory compliance. Compliance frameworks require organizations to demonstrate that documented procedures were followed; however, the informal, ad-hoc documentation practices common during live incidents

Table 1.1.1 - Key Stakeholders in Cybersecurity Incident Response

Role	Responsibility	Operational Pain Points
SOC Analyst (Tier 1/2)	First responder; triages alerts, executes playbook procedures, collects forensic evidence, and escalates as appropriate	Reads static PDF playbooks under acute time pressure; prone to step skipping and incomplete documentation; alert fatigue from high false-positive volumes
SOC Lead / Incident Commander	Supervises and coordinates the response effort; makes real-time escalation and containment decisions; manages inter-team communication	Lacks real-time visibility into step-level response progress; cannot identify bottlenecks until after-action review; dependent on verbal status updates from analysts
CISO / Security Director	Strategic oversight of the IR program; accountable for regulatory compliance; responsible for executive and board-level reporting	Cannot validate the procedural correctness of playbooks before incidents occur; cannot assess coverage gaps across incident types; relies on after-action reports that may omit critical details
Compliance and Audit	Ensures the organization adheres to applicable regulatory frameworks (GDPR, NIS2, SOX, HIPAA); prepares documentation for external auditors	Cannot produce reliable evidence that playbooks were actually followed step-by-step; timeline reconstruction from disparate logs is labor-intensive and error-prone
IT Operations	Maintains infrastructure; performs system isolation, backup restoration, and recovery verification tasks during and after incidents	Receives recovery instructions via informal channels (verbal, email, chat) during crisis conditions; lacks structured, verified recovery procedures
Legal Counsel	Advises on breach notification obligations; assesses litigation exposure; liaises with regulatory bodies	Requires precise timeline of events and response actions for regulatory filings and potential litigation; this data is rarely captured in a structured, defensible format

rarely produce the structured, timestamped, analyst-attributed evidence necessary to satisfy regulatory audit requirements [7].

The second problem is the *playbook quality assurance deficit*. Because playbooks are stored as static documents, there is no systematic mechanism for detecting logical errors, missing steps, undefined variables, or coverage gaps before an incident occurs. Quality control is largely informal and depends on periodic manual review, which is inconsistently performed [1].

These problem areas collectively define the operational and organizational context within which the present project is situated.

1.2 Description of the DSL

The IncidentFlow DSL is proposed as a representational solution to this problem. Instead of treating reporting as a final narrative that is manually composed from scattered operational artifacts, the DSL encodes incident logic directly in a structured, machine-parseable form. The language unifies operational flow and reporting semantics so that required report content emerges from validated process execution [15, 16, 17].

At the model level, the DSL separates reusable context from incident-specific behavior through configuration blocks, imports, and playbook declarations. Within playbooks, the language supports triggers, named phases, conditional branches, iteration, timeout-aware waiting, verification handlers, severity transitions, logging, and explicit control transfer. This design captures the essential response semantics needed for both execution guidance and report reconstruction.

From a usability perspective, the key value is reduction of repetitive manual requirement-checking. If required reporting elements are represented as first-class language constructs (for example, explicit phase boundaries, severity updates, and closure actions), users do not need to repeatedly inspect external templates to verify structural completeness. Completeness becomes a property of the specification and validation pipeline, not a late-stage manual proofreading task.

From an architectural perspective, the DSL provides deterministic syntax and a stable parse-tree representation that can support downstream automation. These properties enable future capabilities such as generated report templates, policy conformance validation, timeline reconstruction, and evidence packaging. In this sense, the DSL is both a domain notation and an integration surface between operations and governance.

Language Architecture

2.1 Grammar Structure

The IncidentFlow grammar is designed as a domain-oriented formal specification that models how incident procedures are described, validated, and prepared for automated report generation [18]. At the top level, the language is intentionally constrained: a valid program can contain configuration blocks, import declarations, and one or more playbook declarations. This creates a predictable document structure and prevents partially defined incident procedures from being accepted as complete specifications.

```
1 program
2   : config_block* import_decl* playbook_decl+ EOF
3   ;
4
5 import_decl
6   : IMPORT STRING (AS IDENTIFIER)?
7   ;
8
9 config_block
10  : CONFIG config_name COLON config_entry+
11  ;
```

These underlying grammar rules dictate the parsing boundary for the DSL structure depicted here. Specifically, the program rule requires that every processed script evaluates optional metadata definitions first (config_block and import_decl), while strictly mandating at least one playbook_decl to form a valid execution graph.

From an architectural perspective, this top-level design enforces a useful separation of concerns. Configuration captures stable, reusable organizational metadata, while playbooks capture incident-specific behavior. The report model then becomes derivable from explicit execution constructs instead of informal prose. This is the first step toward reducing manual requirement-checking and enforcing template completeness through syntax.

The grammar models incident lifecycle behavior with explicit declarations for triggers, phases, and report blocks. A playbook is therefore not a flat script but a structured operational narrative. Triggers define activation conditions, phases encode controlled progression, and report sections capture closure actions.

```
1 playbook_decl
2   : PLAYBOOK IDENTIFIER trigger_decl phase_decl+ report_decl?
```

```

3      ;
4
5 trigger_decl
6     : TRIGGER COLON expr
7     ;
8
9 phase_decl
10    : PHASE IDENTIFIER COLON statement+
11    ;
12
13 report_decl
14    : REPORT COLON statement+
15    ;

```

This structure directly supports reporting automation: each phase contributes traceable execution events, and the report declaration provides explicit closing semantics. Instead of asking users to re-check external templates for required sections, required reporting elements are represented in the grammar itself.

The statement system is expressive enough for realistic incident handling while remaining intentionally limited to domain-relevant constructs. The grammar includes execution actions, conditional paths, iteration, waiting with timeout handling, verification with failure handling, logging, control transfer, assignment, parallel execution intent, and severity control.

```

1 statement
2     : do_stmt
3     | if_stmt
4     | for_stmt
5     | wait_stmt
6     | verify_stmt
7     | log_stmt
8     | goto_stmt
9     | assign_stmt
10    | parallel_block
11    | severity_stmt
12    ;
13
14 wait_stmt
15    : WAIT IDENTIFIER FROM expr TIMEOUT DURATION timeout_handler?

```

```

16      ;
17
18 verify_stmt
19     : VERIFY expr fail_handler?
20     ;

```

Expression rules are organized by precedence to avoid ambiguity and to keep condition semantics deterministic. This is essential for reproducibility and future static analysis.

```

1  expr
2    : or_expr
3    ;
4
5  or_expr
6    : and_expr (OR and_expr)*
7    ;
8
9  and_expr
10   : not_expr (AND not_expr)*
11   ;
12
13 not_expr
14   : NOT not_expr
15   | comparison
16   ;
17
18 comparison
19   : primary (comp_op primary)?
20   ;

```

The literal and access model supports practical policy authoring. In addition to primitive literals, the language supports list literals, member access, and chain literals, which are useful for escalation sequences and organizational routing metadata.

```

1  config_value
2    : STRING
3    | INT
4    | severity_lit

```

```

5      | bool_lit
6      | list_lit
7      | chain_lit
8      ;
9
10 list_lit
11      : LBRACKET config_value (COMMA config_value)* RBRACKET
12      ;
13
14 chain_lit
15      : IDENTIFIER (ARROW IDENTIFIER)+
16      ;

```

Finally, the grammar balances strictness and extensibility. Strictness is provided by explicit keyword-driven forms and mandatory structural elements. Extensibility is provided by modular rules that can evolve without changing core parse entry points. This combination makes the grammar suitable for iterative DSL growth while maintaining compatibility for automated report generation pipelines.

2.2 Lexer Rules

The lexer defines the lexical contract of the language and transforms source text into domain-aware tokens used by the parser. In IncidentFlow, lexical design is intentionally explicit: operational terms are reserved as keywords so the syntax remains self-documenting and less error-prone.

```

1 PLAYBOOK      : 'PLAYBOOK' ;
2 CONFIG         : 'CONFIG' ;
3 TRIGGER        : 'TRIGGER' ;
4 PHASE          : 'PHASE' ;
5 DO             : 'DO' ;
6 IF             : 'IF' ;
7 THEN           : 'THEN' ;
8 ELSE           : 'ELSE' ;
9 WAIT           : 'WAIT' ;
10 TIMEOUT        : 'TIMEOUT' ;
11 ON_TIMEOUT     : 'ON_TIMEOUT' ;
12 VERIFY         : 'VERIFY' ;
13 ON_FAIL        : 'ON_FAIL' ;
14 REPORT         : 'REPORT' ;

```

```
15 SEVERITY      : 'SEVERITY' ;
```

Domain literals are captured with dedicated token rules. The objective is to validate typing constraints as early as possible and avoid postponing basic correctness checks to semantic phases.

```
1 IDENTIFIER    : [a-zA-Z_] [a-zA-Z0-9_]* ;
2 STRING        : '"' (~["\\r\n] | '\\\'' .)* '"' ;
3 DURATION      : [0-9]+ ('sec' | 'min' | 'hr') ;
4 INT           : [0-9]+ ;
5
6 TRUE          : 'TRUE' ;
7 FALSE         : 'FALSE' ;
8 LOW           : 'LOW' ;
9 MEDIUM       : 'MEDIUM' ;
10 HIGH          : 'HIGH' ;
11 CRITICAL      : 'CRITICAL' ;
```

Operator rules are declared with careful ordering to preserve longest-match behavior and avoid tokenization conflicts. This is particularly important for comparison operators where prefix overlap exists.

```
1 EQ            : '==' ;
2 NEQ           : '!=' ;
3 GTE           : '>=' ;
4 LTE           : '<=' ;
5 GT            : '>' ;
6 LT            : '<' ;
7
8 ASSIGN        : '=' ;
9 ARROW         : '->' ;
10 COLON         : ':' ;
11 COMMA        : ',' ;
12 DOT           : '.' ;
13 LPAREN        : '(' ;
14 RPAREN        : ')' ;
15 LBRACKET      : '[' ;
16 RBRACKET      : ']' ;
```

Lexical sanitation is handled through explicit skip rules for comments, whitespace, and BOM markers. This keeps token streams stable across operating systems and editor configurations.

```

1 COMMENT      : '#' ~[\r\n]* -> skip ;
2 BLOCK_COMMENT : '/*' .*? '*/' -> skip ;
3 WHITESPACE    : [ \t\r\n]+ -> skip ;
4 BOM           : '\uFEFF' -> skip ;

```

The lexer therefore contributes directly to usability goals: authors can write concise, readable play-books using domain terms, while the implementation enforces lexical consistency automatically.

2.3 Parser Logic

Parser logic is implemented through generated parser classes and a dedicated runtime driver. The runtime flow follows a standard but robust sequence: source loading, character stream creation, lexical analysis, token stream construction, parser invocation, and parse-tree materialization through the program entry rule.

```

1 CharStream input = CharStreams.fromString(source);
2 IncidentFlowLexer lexer = new IncidentFlowLexer(input);
3 CommonTokenStream tokens = new CommonTokenStream(lexer);
4 IncidentFlowParser parser = new IncidentFlowParser(tokens);
5 ParseTree tree = parser.program();

```

A key design decision is centralized syntax error accounting through a custom listener attached to both lexer and parser. This provides a uniform diagnostic channel and supports fail-fast termination when invalid constructs are present.

```

1 ErrorCounter errors = new ErrorCounter();
2 lexer.removeErrorListeners();
3 lexer.addErrorListener(errors);
4 parser.removeErrorListeners();
5 parser.addErrorListener(errors);
6
7 if (errors.getCount() > 0) {
8     System.err.println("Parsing finished with " + errors.getCount() + " error(s).");
9     System.exit(2);
10 }

```

The error listener implementation is lightweight and deterministic, which is appropriate for build pipelines where syntax validity must be an objective gate.

```

1 private static class ErrorCounter extends BaseErrorListener {

```

```

2     private int count = 0;
3
4     @Override
5     public void syntaxError(Recognizer<?, ?> recognizer, Object offendingSymbol,
6                             int line, int charPositionInLine,
7                             String msg, RecognitionException e) {
8         System.err.printf("line %d:%d %s%n", line, charPositionInLine, msg);
9         count++;
10    }
11
12    int getCount() { return count; }
13 }

```

From a parser architecture viewpoint, three properties are relevant for the DSL objective. First, structural validation is guaranteed before execution or report generation begins. Second, parse trees preserve full syntactic context, enabling deterministic extraction of phase transitions, severity changes, and report actions. Third, parser output can be transformed into intermediate models for downstream generators that produce human-readable reports without requiring users to manually satisfy template constraints.

The generation and execution workflow is automated through a build script that regenerates parser artifacts, compiles runtime classes, and validates a representative DSL sample. This allows grammar evolution to be tested continuously.

```

1 java -jar "${ANTLR_JAR}" \
2     -o gen \
3     -package incidentflow \
4     -visitor \
5     -listener \
6     IncidentFlow.g4
7
8 javac -cp "${ANTLR_JAR}" -d bin $(find gen -name '*.java') Main.java
9 java -cp "bin:${ANTLR_JAR}" incidentflow.Main sample.iflow

```

In summary, parser logic in IncidentFlow is intentionally conservative and reliability-oriented. The implementation favors explicit syntax, deterministic error reporting, and reproducible parse outcomes. These properties are necessary for a DSL whose primary architectural promise is trustworthy automation of incident reporting.

2.4 Derivation Tree Analysis

The derivation tree is the most direct structural representation of how an IncidentFlow specification is recognized by the grammar. Whereas token streams capture lexical units and parser rules define admissible forms, the derivation tree materializes the full hierarchical decomposition from the start symbol to terminal constructs. In this project, the tree is used as an internal parser artifact for grammar validation, semantic mapping, and quality assurance in language evolution.

For readability and layout continuity, the full derivation tree visual is provided in Appendix A.4.

At the root level, the tree is anchored in `program`, which expands into optional configuration and import regions, followed by one or more `playbook_decl` nodes. This root structure is architecturally significant because it enforces a canonical specification order and prevents malformed documents where operational logic exists without declarative context.

The next layer captures playbook identity and activation logic. A `playbook_decl` node expands into the playbook name, a mandatory `trigger_decl`, one or more `phase_decl` blocks, and an optional `report_decl`. In the tree, this pattern makes the lifecycle skeleton explicit: activation condition, staged response behavior, and closure/reporting behavior. The tree therefore encodes not only syntax, but a normalized operational narrative.

Within each phase, statement nodes form deeply nested subtrees that correspond to control-flow semantics. Conditional branches generate `if_stmt` subtrees with explicit `THEN` and optional `ELSE` branches; loops generate `for_stmt` branches over iterable expressions; timeout and failure handling produce nested handler branches under `wait_stmt` and `verify_stmt`. This nesting is essential for downstream transformations, since it preserves execution dependencies and fallback logic that are often lost in flat, manually written reports.

Expression subtrees reveal another critical property: precedence-aware normalization. Nodes for `or_expr`, `and_expr`, `not_expr`, and `comparison` are arranged according to grammar precedence, ensuring that complex trigger and verification predicates are represented unambiguously. For report automation, this is important because condition rationale can be reconstructed from tree structure rather than inferred from free-form text.

The tree also provides a bridge between procedural and reporting semantics. Subtrees containing `LOG`, `SEVERITY`, `VERIFY`, and closure actions in `report_decl` can be mapped to report sections such as timeline events, severity evolution, validation outcomes, and closure summary. In other words, the derivation tree provides a deterministic intermediate representation from which reporting artifacts can be generated with consistent structure.

To illustrate structural depth, the following simplified textual projection shows the core hierarchy

observed in a typical sample parse:

```
1 program
2   |- config_block*
3   |- import_decl*
4   '- playbook_decl+
5       |- PLAYBOOK IDENTIFIER
6       |- trigger_decl
7       |   '- expr
8       |- phase_decl+
9       |   |- PHASE IDENTIFIER
10      |   '- statement+
11          |- if_stmt
12          |- wait_stmt
13          |- verify_stmt
14          |- do_stmt
15          '- severity_stmt
16  '- report_decl?
17      '- statement+
```

From an engineering perspective, this tree-centric view supports three advanced activities. First, it enables regression testing of grammar changes by comparing expected and actual subtree shapes for representative inputs. Second, it allows semantic passes to consume stable node patterns rather than brittle textual heuristics. Third, it improves diagnosability: when parse errors occur, developers can localize failures by identifying which subtree expansion failed and at what structural depth.

Overall, the derivation tree should be viewed as a parser-level intermediate representation, not as the core architecture of the final IncidentFlow product. Its role is to make syntax processing deterministic and verifiable, so that higher-level components can reliably translate authored playbooks into automation-ready reporting representations.

Implementation

This chapter details the design, structure, and implementation of IncidentFlow, describing how the CLI, frontend, and HTTP API work together to enable automated incident response documentation.

The IncidentFlow ecosystem is built upon a modular architecture, separating the core language compilation from the user-facing interfaces. At the parser level, ANTLR leverages the `IncidentFlow.g4` grammar definition to translate the source text into a deeply nested Parse Tree. The sequence of language tokens navigates through the lexer to extract fundamental constructs representing triggers and phases.

The compiler traverses this tree, checking block topologies and identifying invalid state jumps (e.g., verifying that GOTO expressions route securely to instanced blocks). During compilation, the engine tracks concurrent definitions encapsulated in PARALLEL directives, mapping them coherently to report structures rather than raw serialized logging commands. Wait behaviors map cleanly into distinct timeline expectations inside the final Markdown string. The report generation logic fundamentally hinges on a template translation pattern parsing the semantic tree sequentially, generating Markdown header blocks, and rendering an accumulative trail of SEVERITY adjustments.

3.0.1 Command-Line Interface (CLI)

The `iflow` command serves as the primary entry point for interacting with the compiler toolkit. To deploy the toolchain, users can execute the provided `install.sh` script from the project root, which globally registers the wrapper within the local binary paths. Alternatively, it can be executed natively from the source directory via `./iflow <command>`.

Once established, the CLI exposes a diverse suite of workflows tailored to validation, active processing, and interactive generation.

At the foundational validation level, the `check` command evaluates a provided playbook for both syntax compliance against the ANTLR grammar and deeper semantic consistency. For example, executing:

```
1 $ iflow check playbooks/sample.iflow
2   -> Checking playbooks/sample.iflow ...
3   0: No errors found.
```

guarantees that the underlying state machine is well-formed. Crucially, the semantic checks verify constraints such as ensuring all GOTO execution jumps reference strictly existing and valid phases. When clean, the process exits with code 0; otherwise, it predictably terminates with codes 2 or 3 depending on whether it failed syntactically or semantically.

Once a playbook is verified, the `report` command orchestrates the compilation pipeline, translating the technical response script into a structured Markdown document. This generated report encompasses

configuration schemas, phase-by-phase action breakdowns, explicit logging entries, and an accumulative severity evolution trail. By default, running a report prints the Markdown payload directly to the standard output:

```
1 $ iflow report playbooks/phishing.iflow
2   # Incident Report
3   ## Phishing Details...
```

Redirecting this output to an artifact file is natively supported via the `-o` flag (e.g., `iflow report playbooks/phishing.iflow -o out.md`).

To accelerate incident response workflows during high-pressure scenarios, the CLI provides an advanced interactive wizard command. Rather than demanding engineers to write raw DSL code from scratch, starting an interactive session sequentially queries the analyst regarding incident symptoms and evidence. The engine prompts for specific MITRE ATT&CK classifications, currently encompassing T1566 (Phishing), T1059.001 (PowerShell Execution), and T1055 (Process Injection).

```
1 $ iflow wizard
2
3 Select the attack technique:
4   1) T1566      - Phishing
5   2) T1059.001 - PowerShell Execution
6   3) T1055      - Process Injection
7 > 1
8
9 Output base name: > phishing_incident
10
11 What severity level triggered this alert?
12   1) LOW  2) MEDIUM  3) HIGH  4) CRITICAL
13 > 3
```

By feeding numerical prompts back into the console, the wizard transpiles those decisions directly into dual artifacts: a pristine `.iflow` playbook script and a compiled Markdown report.

Beyond standalone task execution, the CLI embeds diagnostic and integration features. Developers can invoke the `tree` command to visually spawn a GUI representing the topological parse tree. This greatly aids grammar diagnostics when extending the incident language:

```
1 $ iflow tree playbooks/sample.iflow
2 -> Launching AST visualizer...
```

For API-first ecosystems, the `serve` command dynamically spawns the HTTP API discussed subsequently. This background service abstracts the CLI layer entirely:

```
1 $ iflow serve -p 8080 -d playbooks/
2   -> Starting HTTP API on port 8080...
3   -> Watching directory: playbooks/
```

This backend accepts target port (`-p`) and active playbook directory (`-d`) arguments, providing a foundational bridge for the React and Vite frontend.

3.0.2 HTTP API Backend

To facilitate interactions from the frontend and external tools, the IncidentFlow compiler operates as a dynamic background service via the `iflow serve` command. It exposes a lightweight JSON REST API routing compilation tasks without the overhead of the CLI environment.

Crucial endpoints include:

- `POST /api/compile`: Accepts arbitrary `.iflow` strings, funneling them through the parser and returning `{ "valid": true }` on success, or structured error arrays on internal failure.
- `POST /api/report`: Runs the validations and appends a `report` JSON key encapsulating the final formatted Markdown string.
- `GET /api/playbooks`: Discovers all pre-existing `.iflow` files allocated in the server's working directory, enabling templates to be queried algorithmically.

A valid code compilation guarantees an HTTP 200 OK. For example, verifying a valid syntax snippet returns a successful boolean mapping:

```
1 // POST /api/compile Request
2 {
3   "code": "PLAYBOOK \"Demo\" \n TRIGGER \"Alert\" \n PHASE Initial {}"
4 }
5
6 // Response (200 OK)
7 {
8   "valid": true,
9   "errors": []
10 }
```

Conversely, malformed source texts yield an HTTP 422 Unprocessable Entity combined with explicit parser notifications detailing the broken token and its precise line or character location:

```
1 // Response (422 Unprocessable Entity)
```

```

2 {
3   "valid": false,
4   "errors": [
5     "line 3:0 mismatched input 'PHASE' expecting 'TRIGGER'"
6   ]
7 }

```

To bridge natively with the visual web interface, CORS (Cross-Origin Resource Sharing) support is compiled directly into the API runtime, enabling decoupled frontend environments to execute HTTP calls directly from the user's browser securely.

3.0.3 Frontend Architecture and UI

The IncidentFlow frontend is built using React and Vite. It serves as a visual client that communicates with the HTTP API for code compilation and validation, eliminating the need to install the CLI locally.

The application layout centers around a text editor for the DSL alongside a real-time Markdown preview window (see Figure 3.1).

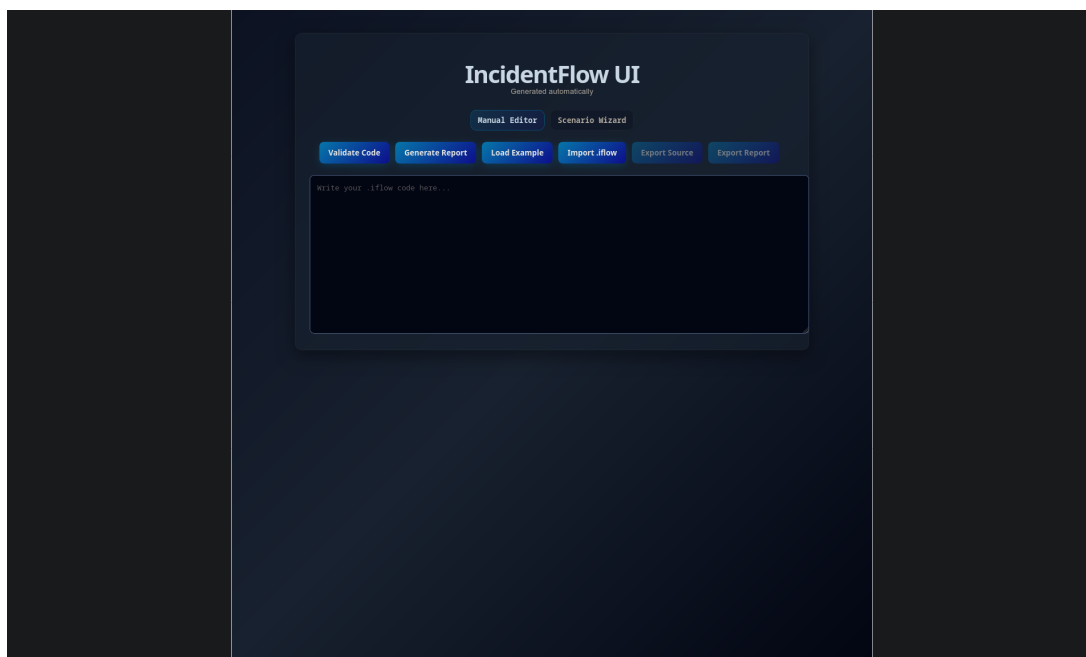


Figure 3.1 - The main editor interface of the IncidentFlow UI.

The application state is managed using standard React hooks. When users type in the Manual Editor, the input is stored in a component state. To avoid overloading the backend with validation requests on every keystroke, a debouncing effect delays the API call until the user pauses typing. The frontend sends the content to the `/api/compile` endpoint and updates the UI based on the response.

Error processing is handled dynamically. When the API returns an HTTP 422 status code, the frontend extracts the error array and renders it below the editor. This translates terminal-based tracebacks into immediate visual feedback:

```
1 <div className="error-banner">
2   Syntax Error at Line 12: Unexpected token 'GOTO'
3 </div>
```

In addition to manual authoring, the frontend implements a visual form to guide users through report generation called the "Scenario Wizard" (illustrated in Figure 3.2).

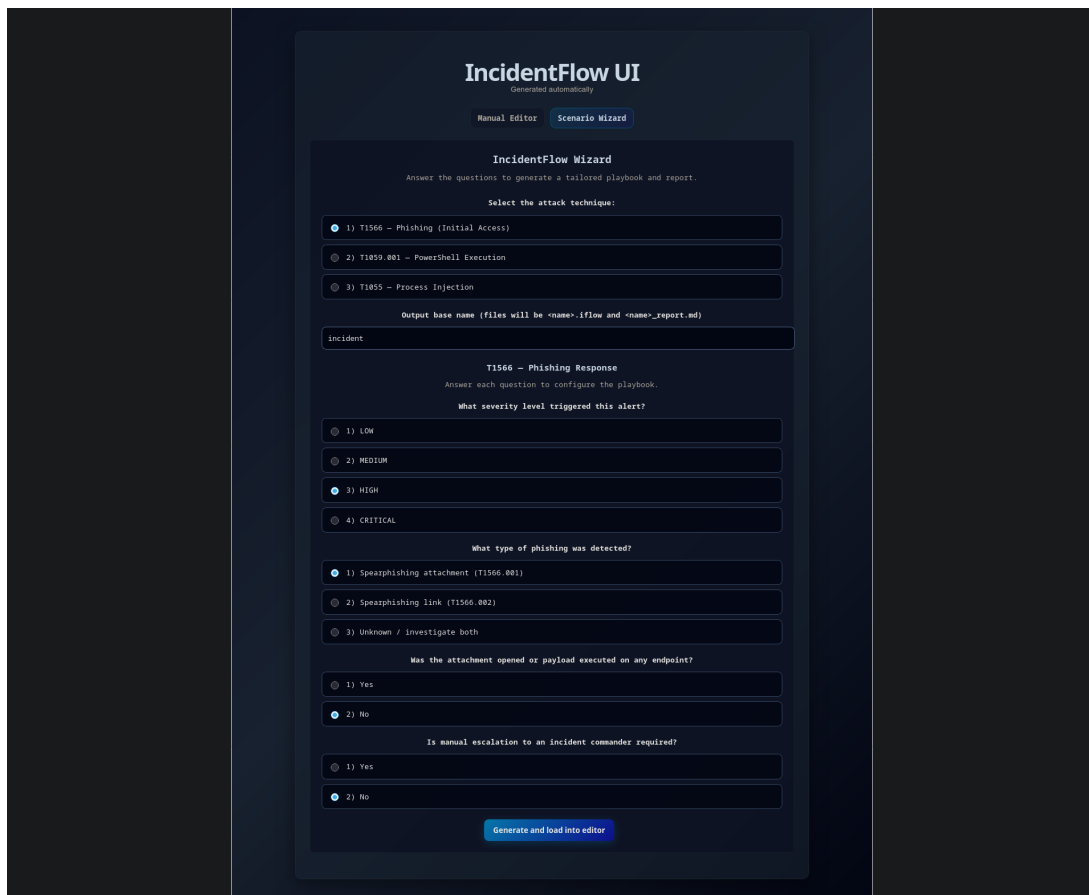


Figure 3.2 - The interactive Scenario Wizard.

This wizard replicates the interactive command-line prompts by iterating through a series of predefined questions regarding the incident, such as MITRE ATT&CK classifications and threat severity. The collected answers are then formatted into valid DSL code and sent directly to the compiler.

For users who prefer modifying existing playbooks, the interface includes a template loader. This feature queries the `/api/playbooks` REST endpoint and presents available scripts in a dropdown menu (shown in Figure 3.3).

Selecting an example overwrites the editor's current state with the loaded string, triggering an auto-

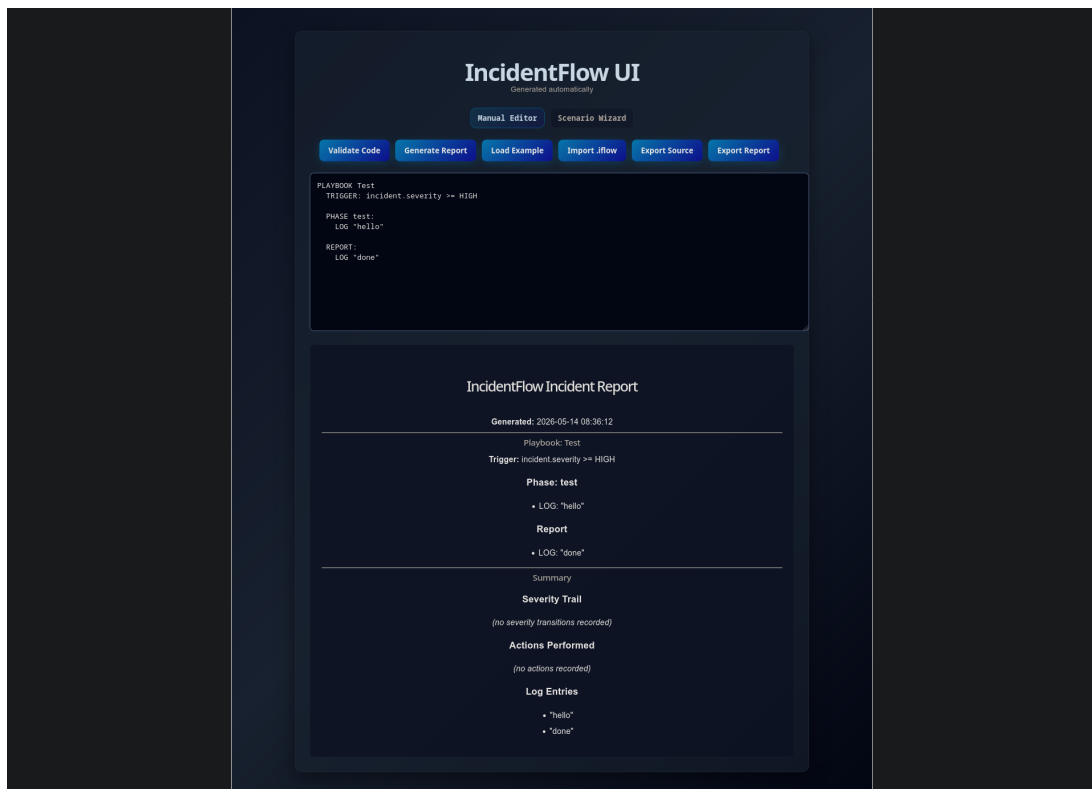


Figure 3.3 - Loading and browsing playbook examples.

matic re-compilation.

Finally, when the DSL code is valid, the frontend requests the final report representation via the `/api/report` endpoint. The JSON response contains a rendered Markdown string. The frontend passes this string to a React Markdown parser, producing a styled HTML document that the user can immediately export or copy (Figure 3.4).

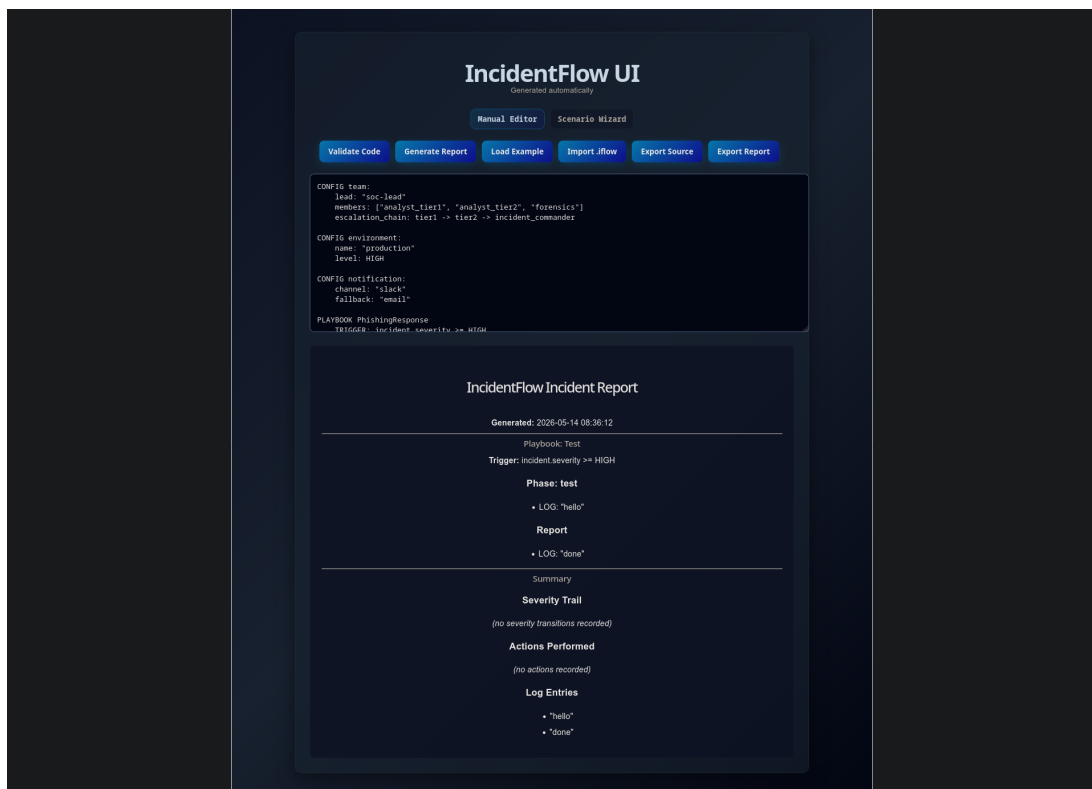


Figure 3.4 - The resultant incident report rendered visually next to the editor.

Conclusions

This project set out to design and implement a domain-specific language (DSL) to partially automate and standardize cybersecurity incident reporting. To achieve the goals outlined in the Abstract, the following objectives were completed:

- **Formal Specification:** A custom grammar (`IncidentFlow.g4`) was implemented using ANTLR, defining language constructs for triggers, phases, logic branching, and severity escalation.
- **Compiler and Validation:** A parser pipeline was established to evaluate the incident playbooks for both syntactic correctness and semantic consistency, effectively replacing manual requirement-checking with automated validation.
- **CLI Toolkit:** A command-line interface was developed to allow engineers to validate scripts and generate structured Markdown incident reports. Specifically, an interactive scenario wizard was added to accelerate playbook generation for common MITRE ATT&CK classifications.
- **API and Frontend Integration:** The core compiler was exposed via an HTTP REST API, which served as the backend for a React and Vite-based web frontend. This enabled real-time syntax checking and report rendering directly in the browser.

The resulting implementation confirms that replacing manual, document-centric incident reporting with a process-centric DSL is technically feasible. The unified architecture successfully bridges the operational necessity of executing technical responses with the administrative requirement of maintaining auditable compliance evidence.

Future development could extend the DSL's capabilities by directly integrating it with enterprise SOAR systems, allowing the compiler to not only generate documentation but simultaneously push automated mitigation commands across network endpoints.

The IncidentFlow DSL project source code relative to this implementation is available at: https://github.com/nnorian/IncidentFlow_dsl/ [19].

Appendix

A.1 Complete Grammar Specification

```
1 grammar IncidentFlow;
2
3 program
4     : config_block* import_decl* playbook_decl+ EOF
5     ;
6
7 import_decl
8     : IMPORT STRING (AS IDENTIFIER)?
9     ;
10
11 config_block
12     : CONFIG config_name COLON config_entry+
13     ;
14
15 config_name
16     : IDENTIFIER
17     ;
18
19 config_entry
20     : IDENTIFIER COLON config_value
21     ;
22
23 config_value
24     : STRING
25     | INT
26     | severity_lit
27     | bool_lit
28     | list_lit
29     | chain_lit
30     ;
31
32 list_lit
```

```

33      : LBRACKET config_value (COMMA config_value)* RBRACKET
34      ;
35
36 chain_lit
37      : IDENTIFIER (ARROW IDENTIFIER)+
38      ;
39
40 playbook_decl
41      : PLAYBOOK IDENTIFIER trigger_decl phase_decl+ report_decl?
42      ;
43
44 trigger_decl
45      : TRIGGER COLON expr
46      ;
47
48 phase_decl
49      : PHASE IDENTIFIER COLON statement+
50      ;
51
52 report_decl
53      : REPORT COLON statement+
54      ;
55
56 statement
57      : do_stmt
58      | if_stmt
59      | for_stmt
60      | wait_stmt
61      | verify_stmt
62      | log_stmt
63      | goto_stmt
64      | assign_stmt
65      | parallel_block
66      | severity_stmt
67      ;
68

```

```

69 do_stmt
70     : DO function_call
71     ;
72
73 if_stmt
74     : IF expr THEN COLON statement+ (ELSE COLON statement+)?
75     ;
76
77 for_stmt
78     : FOR EACH IDENTIFIER IN expr COLON statement+
79     ;
80
81 wait_stmt
82     : WAIT IDENTIFIER FROM expr TIMEOUT DURATION timeout_handler?
83     ;
84
85 timeout_handler
86     : ON_TIMEOUT COLON statement+
87     ;
88
89 verify_stmt
90     : VERIFY expr fail_handler?
91     ;
92
93 fail_handler
94     : ON_FAIL COLON statement+
95     ;
96
97 log_stmt
98     : LOG expr
99     ;
100
101 goto_stmt
102     : GOTO IDENTIFIER
103     ;
104

```

```

105 assign_stmt
106     : IDENTIFIER ASSIGN expr
107     ;
108
109 parallel_block
110     : PARALLEL COLON statement+
111     ;
112
113 severity_stmt
114     : SEVERITY severity_lit
115     ;
116
117 expr
118     : or_expr
119     ;
120
121 or_expr
122     : and_expr (OR and_expr)*
123     ;
124
125 and_expr
126     : not_expr (AND not_expr)*
127     ;
128
129 not_expr
130     : NOT not_expr
131     | comparison
132     ;
133
134 comparison
135     : primary (comp_op primary)?
136     ;
137
138 comp_op
139     : EQ
140     | NEQ

```

```

141     | GTE
142     | LTE
143     | GT
144     | LT
145     ;
146
147 primary
148     : function_call
149     | member_access
150     | IDENTIFIER
151     | literal
152     | LPAREN expr RPAREN
153     ;
154
155 member_access
156     : IDENTIFIER (DOT IDENTIFIER)+
157     ;
158
159 function_call
160     : IDENTIFIER LPAREN argument_list? RPAREN
161     ;
162
163 argument_list
164     : argument (COMMA argument)*
165     ;
166
167 argument
168     : IDENTIFIER COLON expr
169     | expr
170     ;
171
172 literal
173     : STRING
174     | INT
175     | DURATION
176     | bool_lit

```

```

177     | severity_lit
178     | list_lit
179     ;
180
181 bool_lit
182     : TRUE
183     | FALSE
184     ;
185
186 severity_lit
187     : LOW
188     | MEDIUM
189     | HIGH
190     | CRITICAL
191     ;
192
193 PLAYBOOK      : 'PLAYBOOK' ;
194 CONFIG         : 'CONFIG' ;
195 TRIGGER        : 'TRIGGER' ;
196 PHASE          : 'PHASE' ;
197 DO             : 'DO' ;
198 IF             : 'IF' ;
199 THEN           : 'THEN' ;
200 ELSE           : 'ELSE' ;
201 FOR            : 'FOR' ;
202 EACH           : 'EACH' ;
203 IN             : 'IN' ;
204 WAIT           : 'WAIT' ;
205 FROM           : 'FROM' ;
206 TIMEOUT        : 'TIMEOUT' ;
207 ON_TIMEOUT     : 'ON_TIMEOUT' ;
208 ON_FAIL        : 'ON_FAIL' ;
209 VERIFY        : 'VERIFY' ;
210 LOG            : 'LOG' ;
211 GOTO           : 'GOTO' ;
212 REPORT         : 'REPORT' ;

```

```

213 AND          : 'AND' ;
214 OR           : 'OR' ;
215 NOT          : 'NOT' ;
216 PARALLEL     : 'PARALLEL' ;
217 SEVERITY      : 'SEVERITY' ;
218 IMPORT        : 'IMPORT' ;
219 AS            : 'AS' ;
220 TRUE          : 'TRUE' ;
221 FALSE         : 'FALSE' ;
222 LOW           : 'LOW' ;
223 MEDIUM        : 'MEDIUM' ;
224 HIGH          : 'HIGH' ;
225 CRITICAL      : 'CRITICAL' ;
226
227 IDENTIFIER    : [a-zA-Z_][a-zA-Z0-9_]* ;
228 STRING        : '"' (~["\\r\n] | '\\\'.)* '"' ;
229 DURATION       : [0-9]+ ('sec' | 'min' | 'hr') ;
230 INT           : [0-9]+ ;
231
232 EQ            : '==' ;
233 NEQ           : '!=' ;
234 GTE           : '>=' ;
235 LTE           : '<=' ;
236 GT            : '>' ;
237 LT            : '<' ;
238
239 COLON         : ':' ;
240 COMMA         : ',' ;
241 DOT           : '.' ;
242 LPAREN        : '(' ;
243 RPAREN        : ')' ;
244 LBRACKET      : '[' ;
245 RBRACKET      : ']' ;
246 ASSIGN        : '=' ;
247 ARROW         : '->' ;
248

```

```

249 COMMENT      : '#' ~[\r\n]* -> skip ;
250 BLOCK_COMMENT : '/*' .*? '*/' -> skip ;
251 WHITESPACE    : [ \t\r\n]+ -> skip ;
252 BOM           : '\uFEFF' -> skip ;

```

A.2 Parser Driver Excerpt

```

1 public class Main {
2
3     public static void main(String[] args) throws IOException {
4         if (args.length < 1) {
5             System.err.println("Usage: Main <source-file.iflow>");
6             System.exit(1);
7         }
8
9         String source = new String(Files.readAllBytes(Paths.get(args[0])));
10
11         CharStream input = CharStreams.fromString(source);
12         IncidentFlowLexer lexer = new IncidentFlowLexer(input);
13         CommonTokenStream tokens = new CommonTokenStream(lexer);
14         IncidentFlowParser parser = new IncidentFlowParser(tokens);
15
16         ErrorCounter errors = new ErrorCounter();
17         lexer.removeErrorListeners();
18         lexer.addErrorListener(errors);
19         parser.removeErrorListeners();
20         parser.addErrorListener(errors);
21
22         ParseTree tree = parser.program();
23
24         if (errors.getCount() > 0) {
25             System.err.println("\nParsing finished with " + errors.getCount() + " error(s)");
26             System.exit(2);
27         }
28
29         Future<JFrame> dialog = Trees.inspect(tree, parser);

```

```

30     try {
31         dialog.get();
32     } catch (Exception e) {
33         System.err.println("Tree viewer error: " + e.getMessage());
34     }
35 }
36 }

```

A.3 Error Listener Implementation

```

1 private static class ErrorCounter extends BaseErrorListener {
2     private int count = 0;
3
4     @Override
5     public void syntaxError(Recognizer<?, ?> recognizer, Object offendingSymbol,
6                             int line, int charPositionInLine,
7                             String msg, RecognitionException e) {
8         System.err.printf("line %d:%d %s%n", line, charPositionInLine, msg);
9         count++;
10    }
11
12    int getCount() { return count; }
13 }

```

A.4 Derivation Tree Figure



A.5 Playbook Sample

```
1 CONFIG team:
2     lead: "alice"
3     members: ["bob", "carol", "dave"]
4     escalation_chain: tier1 -> tier2 -> oncall
5
6 CONFIG environment:
7     name: "production"
8     level: HIGH
9
10 IMPORT "policies/escalation.iflow" AS ep
11
12 PLAYBOOK DatabaseOutage
13     TRIGGER: incident.severity >= HIGH
14
15     PHASE triage:
16         status = pingDB()
17         VERIFY checkConnection()
18         ON_FAIL:
19             SEVERITY CRITICAL
20             LOG "Primary DB unreachable - escalating"
21             GOTO escalate
22
23         IF status == FALSE THEN:
24             DO sendAlert(channel: "oncall", message: "DB down")
25             SEVERITY HIGH
26         ELSE:
27             LOG "Connectivity check passed"
28
29     PHASE investigate:
30         FOR EACH node IN db.nodes:
31             result = healthCheck(node: node)
32             IF result == FALSE THEN:
33                 LOG "Unhealthy node detected"
34                 DO notifyPager(user: node.owner, priority: CRITICAL)
```

```
35
36     WAIT ack FROM oncall TIMEOUT 15min
37     ON_TIMEOUT:
38         SEVERITY CRITICAL
39         GOTO escalate
40
41 PHASE escalate:
42     PARALLEL:
43         DO triggerPolicy(level: CRITICAL)
44         DO takeSnapshot(scope: "db", format: "full")
45     SEVERITY CRITICAL
46     LOG "Escalation policy triggered"
47
48 REPORT:
49     LOG "Incident closed"
50     SEVERITY LOW
51     DO closeTicket(id: incident.id, resolution: "auto")
```

A.6 Problem Map

a)	Present the problem situation in 5-7 sentences.	When a cybersecurity incident occurs such as a ransomware attack, data breach, or brute force intrusion organizations rely on incident response playbooks to guide their teams through detection, containment, eradication, and recovery. Currently, these playbooks exist as static Word documents, PDFs, or wiki pages that team members must read and follow manually under extreme time pressure. This manual approach leads to skipped steps, miscommunication, delayed escalations, and inconsistent responses. There is no way to test or validate a playbook before a real incident occurs. Existing automation tools SOAR platforms address this through drag-and-drop visual interfaces, but these are vendor-locked, expensive, and cannot be version-controlled, peer-reviewed, or tested like code. Security teams need a way to express their response procedures in a format that is both human-readable and machine-executable, bridging the gap between policy documentation and operational automation
b)	Formulate the problem – 1 statement.	Incident response playbooks are currently static, untestable documents that cannot be validated or executed automatically, leading to slow, inconsistent, and error-prone responses during cybersecurity incidents
TYPE OF QUESTION	EXAMPLES OF QUESTIONS	SHORT ANSWER
Who...?	1. Who has a problem? 2. Who is involved in the problem situation? 3. Who suffers from the problem? 4. Who should take part in solving the problem?	1. Security Operations Center (SOC) teams, incident responders, and CISOs who are responsible for handling cybersecurity incidents

		2. SOC analysts, incident response leads, forensics specialists, IT administrators, compliance officers, and management who approve and audit response procedures 3. The entire organization because delayed or inconsistent incident response leads to greater financial damage, data loss, regulatory penalties, and reputational harm 4. Security engineers, DSL/language designers, and domain experts in incident response who can define the correct playbook structures and validate the language design
What ...? Which...?	5. What happened? 6. What is the problem? 7. What led you to identify the problem (reasons, causes, evidence, arguments, findings, survey results, etc.)? 8. What are the causes that affect those involved? 9. What are the needs of those targeted? 10. What are the resources (financial, human, logistical, time, etc.) necessary to solve the problem?	5. Incident response procedures remain trapped in static documents while every other part of the security pipeline like detection, monitoring, alerting has been automated. 6. There is no dedicated language that allows IR teams to write, validate, and execute incident response playbooks as code. 7. Analysis of existing IR workflows shows that playbooks from organizations like CISA and CERT Société Générale are distributed as PDFs and Word documents. Industry reports indicate that the average breach takes over 100 days to identify and remediate, partly due to manual, unstructured response processes

		8. Lack of automation tools accessible to non-developers, reliance on static documentation, absence of pre-execution validation, vendor lock-in of existing SOAR platforms, and high cost of commercial solutions 9. A simple, readable, domain-specific language that allows writing playbooks in familiar IR terminology, with static validation, automated execution, team configuration, and report generation without requiring programming expertise 10. A team of 4 developers, one semester of development time, Python as the host language, the Lark parser library, and access to publicly available playbook templates for reference and testing
Where...?	11. Where did the problem arise (location, area, stage of the process of the problem arising)?	11. In Security Operations Centers and incident response teams across organizations of all sizes, where the gap between written policy and operational execution remains unaddressed
When...?	12. When did the problem arise? 13. When was the problem identified? 14. When is it planned to be resolved?	The problem has existed since organizations began formalizing incident response procedures, but has become critical as cyberattacks have grown in frequency and sophistication over the past decade During the analysis phase of this project, when reviewing how real-world IR teams operate and comparing their static playbook documents against the automation

		capabilities available in other security domains By the end of the current semester, delivering a functional DSL prototype with parser, validator, interpreter, and example playbooks
Why...?	15. Why is there a need to solve the problem?	15. Because manual incident response is too slow and error-prone for modern cyber threats. Faster, validated, and automated response directly reduces breach impact, financial loss, and recovery time
How...?	16. How (in what way) can the problem be solved? 17. How will the elimination of the problem be determined?	16. By designing and implementing IncidentFlow a domain-specific language that allows IR teams to write playbooks as executable code with phases, triggers, conditionals, escalation chains, and automated report generation 17. By demonstrating that playbooks written in IncidentFlow can be statically validated for correctness, executed against simulated incidents, and produce accurate timeline reports all without requiring the user to know a general-purpose programming language
How much...?	18. How big is the problem? 19. How much is the quantity/quality/number of people etc. affected? 20. How long will it take to resolve the problem? 21. How many parties need to be involved in resolving it?	18. How big is the problem? It affects virtually every organization with a cybersecurity function. Even large enterprises with SOAR tools struggle with playbook maintenance and validation. 19. Thousands of SOC teams globally. According to industry

		data, the cybersecurity workforce gap exceeds 3.5 million people 20.The prototype will be developed over one semester 21.A team of 4 members covering language design, parser implementation, execution engine, validation, testing, and documentation
Other questions you found answered (optional)	22. 23.	-
	How do you see solving this problem (in terms of deliverable/product/result)?	a working DSL with a parser, static validator, simulated execution engine, and report generator, accompanied by three example playbooks and a comprehensive project report documenting the design and implementation
	Conclusions, findings, reflections...	Current incident response practices rely heavily on human interpretation of static documents under high-stress conditions. While SOAR platforms partially address this, they are expensive, vendor-locked, and GUI-based making them impossible to version-control or peer-review

PBL PROBLEM MAP



Team Contract

IncidentFlow: A Domain-Specific Language for Cybersecurity Incident Reporting

Semester: 4

Project theme: Cybersecurity DSL for incident reporting

Project period: 26.01.2026 - 20.06.2026

ECTS: 8

Supervisory team: Supervisors: Cojuhari Irina; Mentors: Brînzan Leon

Project group number: 17

All team members agree to actively and equally contribute to the development of the IncidentFlow project. Each member is responsible for participating in research, design, implementation, and documentation tasks.

We agree to:

1. Attend and actively participate in regular team meetings.
2. Communicate clearly and respectfully with all team members.
3. Complete assigned tasks on time and maintain consistent progress.
4. Collaborate on coding, testing, and writing all sections of the project.
5. Review each other's work and provide constructive feedback.
6. Use shared tools (e.g., version control systems) responsibly and consistently.
7. Make decisions collectively, based on group discussion and majority agreement.

Each member accepts full responsibility for the overall quality and completeness of the project, not only their individual contributions. Failure to follow these agreements may result in task reassignment, reduced individual evaluation, or escalation to the project supervisor.

Botezatu Gabriela, FAF-243

Kushnirenko Ekaterina, FAF-243

Ojog Felicia, FAF-243

Șoimu Ionuț, FAF-243

By signing this document, each member of the group confirms participation on equal terms in the process of writing the project. Thus, each member of the group is responsible for the all contents in the project.

Chisinau, 2026

A.8 Project Plan

The purpose of this project is to address a key limitation in cybersecurity incident response: the reliance on static, document-based playbooks that cannot be validated or executed automatically. As identified in the problem analysis, incident response procedures are typically stored as PDF, Word, or wiki documents and must be manually followed under time pressure, leading to skipped steps, miscommunication, and inconsistent results.

The objective of the project is to develop a structured approach for incident handling that improves consistency, reduces human error, and enables automated validation and execution of response procedures. The expected outcome is a working prototype capable of representing incident workflows in a structured form, validating them, and supporting automated reporting.

The development process follows an iterative methodology. It begins with domain analysis and problem identification in the cybersecurity incident response domain. This is followed by the design of a formal language for describing incident procedures. The implementation phase includes the development of a grammar, lexer, and parser, forming a complete parsing pipeline. Finally, the system is tested using representative scenarios to validate correctness and usability.

The project is implemented using ANTLR for parser generation and Java as the primary programming language. Version control is maintained using GitHub, allowing collaboration, version tracking, and reproducibility. Publicly available incident response playbooks are used as references for testing and validation.

The project timeline spans one academic semester. The problem has been identified during the analysis phase of the project, and the solution is planned to be completed by the end of the current semester, resulting in a functional prototype that includes a parser, validator, interpreter, and example playbooks.

Task distribution is shared equally among all team members. Each member participates in all phases of the project, including domain analysis, language design, implementation, testing, and documentation. This ensures equal contribution, shared responsibility, and a consistent understanding of all components of the system.

The expected results of the project include a functional prototype capable of parsing and validating structured incident descriptions and generating consistent reporting outputs. The system aims to improve the speed, accuracy, and reliability of incident response by reducing reliance on manual documentation and enforcing structure through formal specification.

The team operates under a formal agreement that ensures equal participation and shared responsibility among all members. Each member commits to contributing to all aspects of the project, respecting deadlines, and maintaining effective communication. The team contract, included in the appendix of the

report, defines responsibilities and penalties for non-compliance, ensuring accountability throughout the project.

3.1 Additional UI Views

Bibliography

- [1] Kevin Mandia, Chris Prosise, and Matt Pepe. *Incident handlers handbook*. English. Tech. rep. Bethesda, MD, USA: SANS Institute, 2021, p. 89. URL: <https://www.sans.org/white-papers/33901/>.
- [2] IBM Security. *Cost of a data breach report 2025*. English. Tech. rep. Armonk, NY, USA: IBM Corporation, 2025, p. 45. URL: <https://www.ibm.com/reports/data-breach>.
- [3] Paul Cichonski, Tom Millar, Tim Grance, and Karen Scarfone. *Computer security incident handling guide (SP 800-61 Rev. 3)*. English. Tech. rep. NIST Special Publication 800-61 Rev. 3. Gaithersburg, MD, USA: National Institute of Standards and Technology, 2025, p. 112. URL: <https://csrc.nist.gov/pubs/sp/800/61/r3/final>.
- [4] Charles Linford. *NIST SP 800-61 Rev. 3: New incident response lifecycle*. English. Tech. rep. Linford & Company, 2025. URL: <https://linfordco.com/blog/nist-sp-800-61/>.
- [5] International Organization for Standardization. *ISO/IEC 27035: Information security, cybersecurity and privacy protection – Managing information security incidents*. English. Geneva, Switzerland: ISO/IEC, 2023.
- [6] Ponemon Institute. *The state of the SOC report 2022*. English. Tech. rep. Traverse City, MI, USA: Ponemon Institute, 2022, p. 32.
- [7] ISACA. *State of cybersecurity 2021*. English. Tech. rep. Schaumburg, IL, USA: ISACA, 2021, p. 56.
- [8] Gartner Research. *Market guide for security orchestration, automation and response solutions*. English. Tech. rep. Stamford, CT, USA: Gartner, 2023, p. 24.
- [9] CloudSEK Research. *10 best SOAR platforms in 2026*. English. Tech. rep. CloudSEK, 2026. URL: <https://www.cloudsek.com/knowledge-base/best-soar-platforms>.
- [10] Shuffle Team. *Shuffle: Open-source SOAR platform*. English. Shuffle, 2026. URL: <https://shuffle.io/>.
- [11] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 (General Data Protection Regulation)*. English. 2016. URL: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- [12] European Parliament and Council of the European Union. *Directive (EU) 2022/2555 (NIS2)*. English. 2022. URL: <https://eur-lex.europa.eu/eli/dir/2022/2555/oj>.
- [13] U.S. Congress. *Sarbanes-Oxley Act of 2002*. English. 2002.
- [14] U.S. Congress. *Health Insurance Portability and Accountability Act of 1996*. English. 1996.

- [15] Christopher Stelly and Vassil Roussev. “Language-based integration of digital forensics and incident response”. English. In: *Proceedings of the 14th International Conference on Availability, Reliability and Security (ARES 2019)*. ACM, 2019, Article 73, 1–8. DOI: [10.1145/3339252.3339278](https://doi.org/10.1145/3339252.3339278).
- [16] Spiros Mouratidis and Andrew Fish. “Modelling language for cyber security incident handling”. English. In: *Computers & Security* 131 (2023), p. 103139. DOI: [10.1016/j.cose.2023.103139](https://doi.org/10.1016/j.cose.2023.103139).
- [17] Christopher Stelly. “A domain specific language for digital forensics and incident response”. English. PhD thesis. New Orleans, LA, USA: University of New Orleans, 2019, p. 145. URL: <https://scholarworks.uno.edu/td/2706/>.
- [18] Terence Parr. *The definitive ANTLR 4 reference*. English. Raleigh, NC, USA: Pragmatic Bookshelf, 2013.
- [19] Ekaterina Kushnirenko, Felicia Ojog, Gabriela Botezatu, and Ionut Soimu. *IncidentFlow_dsl: Source Code Repository*. English. 2025. URL: https://github.com/nnorian/IncidentFlow_dsl/.